

Small Language Model Agents Enable Efficient and High-Quality Knowledge Mining

Sipeng Zhang
siz018@ucsd.edu
University of California, San Diego
San Diego, California, USA

Yihang Chen
ychen3726@gatech.edu
Georgia Institute of Technology
Atlanta, Georgia, USA

Shuhuai Lin
shuhuil@andrew.cmu.edu
Carnegie Mellon University
Pittsburgh, Pennsylvania, USA

Pin Qian
pqian@alumni.cmu.edu
Carnegie Mellon University
Pittsburgh, Pennsylvania, USA

Xinpeng Wei
william.xp.wei@outlook.com
Georgia Institute of Technology
Atlanta, Georgia, USA

Su Wang
suwang@alumni.cmu.edu
Carnegie Mellon University
Pittsburgh, Pennsylvania, USA

Huan Xu
huan.xu71@gmail.com
Georgia Institute of Technology
Atlanta, Georgia, USA

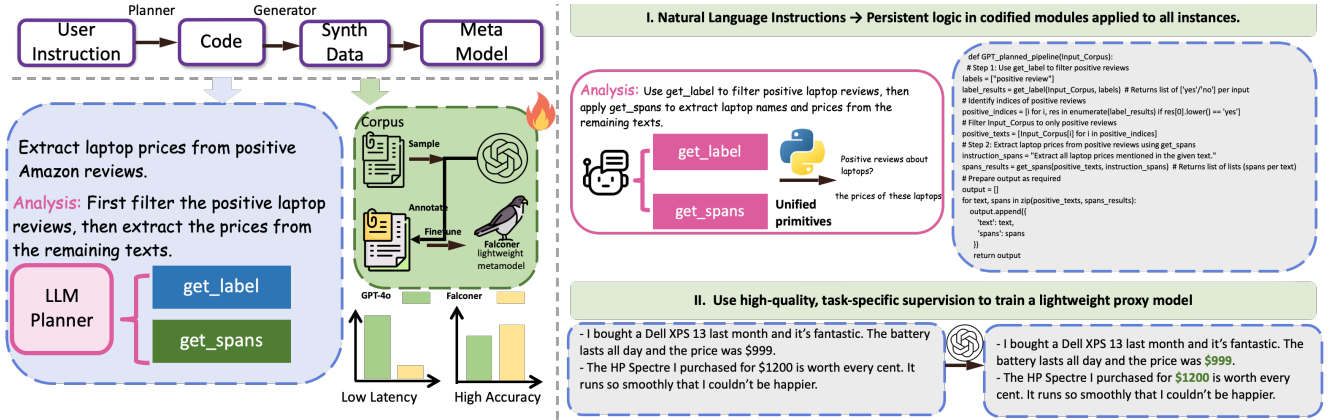


Figure 1: Falconer decomposes the instruction *Extract all laptop prices from positive Amazon reviews* into the primitives `get_label` and `get_span`, generates supervision to train proxy models, and executes these primitives efficiently via small-model inference. On the right, we illustrate how Falconer instantiates the subtasks: it first classifies reviews as positive and then extracts the corresponding price spans. This design combines the instruction-following capability of LLMs with the efficiency of small models.

Abstract

Large language models (LLMs) exhibit strong instruction-following, planning, and reasoning capabilities, enabling new paradigms for knowledge mining—for example, Deep-Research-style extraction of labels and spans from massive corpora based on simple natural-language instructions. However, invoking LLMs at corpus scale for each classification or extraction instance is prohibitively expensive. In contrast, small models and classic pipelines are efficient but difficult to retarget and often brittle under distribution shifts. We present Falconer, a framework that combines the planning capability of LLMs with the efficiency of traditional data mining models (e.g., BERT). Falconer uses an LLM to synthesize an executable workflow and generate supervision, while offloading most inference to lightweight proxy models. These proxies expose two primitives, `get_label` and `get_span`, allowing a single instruction-following proxy to replace many task-specific classifiers and extractors. We

further introduce new benchmarks that evaluate both planning and end-to-end knowledge mining. Across tasks, Falconer matches strong LLM baselines while reducing inference cost by up to 90% and accelerating large-scale mining by over 20×, making Deep-Research-style extraction practical at scale.

CCS Concepts

• Information systems → Structured text search; Data mining.

Keywords

Knowledge Mining, Scalable AI

1 Introduction

Knowledge mining operates over massive text corpora to extract structured information that satisfies user-defined instructions [6,

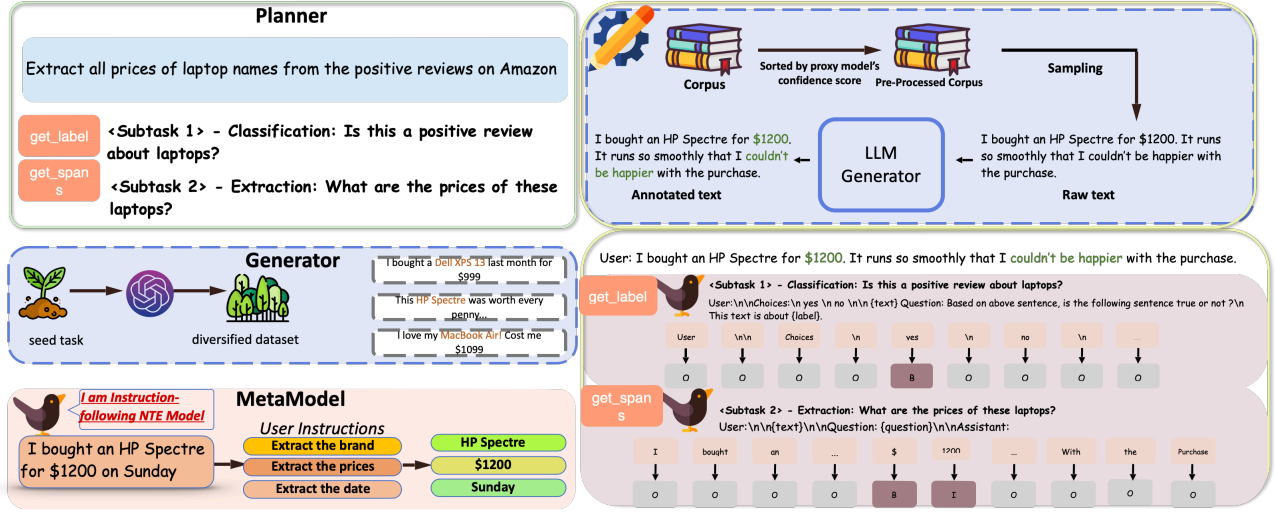


Figure 2: Components Overview

21, 33, 37, 42]. It underpins a wide range of applications in industry, science, and technology; representative examples include parsing customer reviews, analyzing biomedical literature, and summarizing large collections of technical documents [5, 10, 17, 25, 32]. Because these workloads often span millions of records, efficiency is a first-order requirement: systems must sustain high throughput and low cost while maintaining accurate and reliable outputs.

Large language models (LLMs) offer powerful instruction-following and reasoning capabilities, achieving high accuracy on knowledge mining tasks [1, 2, 9, 22, 36, 39]. Yet using LLMs as direct executors is computationally prohibitive: each inference incurs substantial latency and cost, and scaling them to millions of documents quickly becomes infeasible. Conversely, modern small language models (SLMs) deliver excellent inference efficiency and low deployment cost, but their limited reasoning and weak adaptability lead to poor performance on real-world knowledge mining workloads.

This mismatch motivates a hybrid design. We introduce Falconer, a framework that combines the instruction understanding and decomposition strengths of LLMs with the execution efficiency of purpose-built small models. Rather than executing knowledge mining pipelines end-to-end with LLMs, Falconer uses LLMs sparingly in two complementary roles. First, as planners, they translate natural-language task descriptions into structured workflow code that specifies how SLM-based components should be invoked. Second, as annotators, they generate supervision to train lightweight proxy models that implement the workflow efficiently at scale. Once trained, these proxies replace LLMs as the primary executors, enabling throughput suitable for large corpora.

In Falconer, knowledge mining workflows are built from two primitive operations—(1) `get_label(text, instruction)` for classification and (2) `get_span(text, instruction)` for span extraction. These primitives are the atomic calls that SLM-based proxies are optimized to execute. LLM planners compose these primitives into executable pipelines. For example, the instruction

“Extract all laptop prices from positive Amazon reviews” becomes the workflow shown in Figure 1.

More complex tasks, such as relation extraction, entity linking, or multi-hop queries, are expressed as similarly structured compositions. By delegating planning and supervision to LLMs and execution to optimized small models, Falconer achieves both high accuracy and scalable throughput.

This abstraction enables Falconer to unify pipeline construction and execution within a single instruction-following proxy model, eliminating the need for separately trained, task-specific IE components and enabling training paradigms tailored to knowledge-mining workloads. In our implementation, we use Cuckoo [24], a lightweight instruction-following model that is the first to be pre-trained specifically for information extraction under the next-token-extraction (NTE) paradigm. NTE allows Cuckoo to support both `get_label` and `get_span` within one model while following natural-language instructions without predefined label sets or schema-specific engineering. Remarkably, despite its small size, Cuckoo delivers stronger IE performance than LLM-based extractors more than 20× larger, making it an efficient execution engine for Falconer and a unified, adaptive alternative to brittle, hand-crafted pipelines.

To evaluate instruction-following knowledge mining, we introduce new benchmarks that assess both planning quality and end-to-end execution. These benchmarks measure consistency between proxy outputs and annotations from humans and large models. Experiments show that while LLMs remain strong planners, their scalability is limited in practice. In contrast, Falconer closely matches state-of-the-art LLMs in accuracy while reducing inference cost by up to 90% and accelerating large-scale processing by over 20x, making Deep Research style knowledge mining practical at scale.

In summary, our main contributions are threefold:

- We propose Falconer, a framework in which LLMs serve as planners and annotators: they decompose natural-language instructions into executable pipelines and generate supervision for training purpose-built lightweight proxies for knowledge mining.
- We introduce a specifically designed **instruction-following proxy** that unifies classification and extraction as two atomic operations (`get_label`, `get_span`), enabling a single small model to replace multiple task-specific components. We further show that autoregressive small models exhibit limited generalization and adaptation on knowledge-mining-style NER tasks.
- We construct new **instruction-following benchmarks** for knowledge mining that evaluate both planning and end-to-end execution. Experiments show that Falconer closely tracks state-of-the-art LLMs while reducing inference cost by up to 90% and accelerating large-scale processing by over 20×.

2 Related Works

Information Extraction. Information extraction (IE) is one of the most fundamental applications in knowledge mining. IE systems take the user’s requirement (e.g., defined by a label text, a question, or an instruction) and extract spans of several tokens from input texts. IE encompasses a wide range of task formulations with different level of difficulties, which varies from simple structure entity and relation extraction such as named entity recognition [27], relation extraction [7], and event extraction [34], to more difficult tasks such as abstrac entity extraction [26, 41].

LLM Agents. Recent work leverages the advanced reasoning and comprehension abilities of large language models (LLMs) to tackle diverse downstream tasks [4, 31, 43]. For complex scenarios, LLMs have been framed as autonomous agents that interact with environments [8, 19, 44], employ external tools [11, 23, 38, 48], and accumulate experiential knowledge [12, 46]. A representative example is ReAct [44], which tightly integrates reasoning and action by alternating between intermediate reasoning and external operations such as information retrieval.

LLM Agents for Retrieval. LLM agents have been applied to Information Retrieval (IR) through pretraining, reranking, and prompting [29, 35, 47]. As retrievers directly impact downstream tasks such as retrieval-augmented generation [16] and knowledge-intensive QA, domain-specific agents like EHRAgent [30] have been developed to incorporate structured tool-use planning process and an interactive coding mechanism. Nevertheless, existing approaches largely depend on heuristic prompts or few-shot examples, providing limited guidance for effective retrieval strategies and tool-assisted actions.

3 Method

3.1 Preliminary

3.1.1 Background. Modern large language models (LLMs) exhibit strong instruction-following capabilities and generalize well across downstream tasks. However, they scale poorly for knowledge mining workloads that require processing large volumes of data. Traditional knowledge mining systems improve efficiency by chaining specialized classifiers and extractors (e.g., named entity recognition models), but they lack the instruction-following flexibility of LLMs and therefore require developers to manually construct rigid,

Method	Basic	Query-Based	Multi-Entity	Misc.	Misc. w/ ICL
Falconer w/ GPT-4.1	0.96	1.00	1.00	0.21	0.96
Falconer w/ GPT-4o	0.63	0.78	1.00	0.19	0.84
Falconer w/ Claude 3.7 Sonnet	0.78	0.80	0.98	0.19	0.92
Falconer w/ GPT-4o-mini	0.50	0.19	0.30	0.00	0.42

Table 1: Planning correctness score with different LLM as planner.

task-specific pipelines. Recent agentic frameworks aim to balance efficiency with adaptability; nevertheless, they often rely on autoregressive small models as executors, which can limit generalization on tasks that require fine-grained, high-quality token-level supervision (e.g., knowledge mining and NER).

3.1.2 Notations. We briefly describe how different model families perform knowledge mining. Our goal is to extract informative spans from an input sequence $[x_1, x_2, \dots, x_N]$. BIO-style token classification models predict a tag sequence $[y_1, y_2, \dots, y_N]$, where each $y_i \in \{B, I, O\}$. In contrast, autoregressive models typically generate an output sequence $[x_{N+1}, x_{N+2}, \dots, x_{N+K}]$, where K is the number of generated tokens and $x_{N+1}, \dots, x_{N+K} \in \{x_1, x_2, \dots, x_N\}$ correspond to extracted spans.

3.2 Falconer

Our framework consists of three core components: (1) a planner that synthesizes an executable Python function to process a corpus and produce structured outputs in a user-specified format; (2) a generator that produces high-quality supervision for task adaptation; and (3) a compact proxy metamodel that nonetheless performs robustly across diverse tasks. An overview is provided in Figure 1. Formally, given a user prompt $\text{Prompt}_{\text{user}}$ and a corpus Corpus , Falconer extracts text according to $\text{Prompt}_{\text{user}}$ as

$$\text{ExtractedText} = \text{Falconer}(\text{Prompt}_{\text{user}}, \text{Corpus}).$$

Our framework takes the task prompt and output specification, uses the planner to generate execution code, and then leverages the generator and metamodel to produce an adapted proxy for execution. This yields a fully automated pipeline in which users provide a corpus and obtain high-quality outputs, achieving substantial speedups and cost reductions relative to direct LLM execution while maintaining strong performance.

3.3 Unified TC/IE Proxy

The original Cuckoo model is specialized for information extraction (IE) tasks, including basic IE (e.g., entity and relation extraction), query-based IE, and instruction-following IE [24]. Leveraging Cuckoo’s instruction-following capability, we further extend it to text classification via tailored prompt templates. Specifically, we cast text classification as a natural language inference (NLI) problem, where the model predicts whether an input sentence entails a candidate label, as illustrated in Figure 2.

Given a classification task that maps an input sequence $[x_1, x_2, \dots, x_N]$ to one of m classes $\{C_1, C_2, \dots, C_m\}$, we use the instruction-following ability of BIO-tagging models and format the input as ["Choice",

": ", "Yes", "No", $x_1, x_2, \dots, x_N$, "SystemPrompt", C] where where $C \in \{C_1, C_2 \dots C_m\}$ is a candidate class and SystemPrompt is a fixed template (e.g., "Question: Based on the above sentence, is the following sentence true or not? This text is about"). To this end, we construct an instruction-based prompt template for classification and fine-tune Super Rainbow Cuckoo on the datasets introduced in Laurer et al. [15], yielding the metamodel used in our experiments.

3.4 Planning

The planner is the core of Falconer, translating natural language requirements into executable pipelines by codifying instructions into atomic operations and explicit control flows. For a knowledge mining objective, it decomposes the input into subtasks (e.g., classification, span extraction), each bound to a tool interface such as `get_label` or `get_span`. These are then assembled into a deterministic control flow, ensuring explicit execution without reliance on implicit reasoning. Specifically, we prompt modern, high-capacity large language models to explicitly decompose each task into two operations `get_label` and `get_span`, which are internally instantiated by the proxy model, as illustrated in Figure 2. Sample code is shown in Figure 1 and Appendix E.

Crucially, the planner does not merely synthesize runnable code but codifies the logical dependencies among subtasks. For example, in a multi-entity extraction scenario, *Retrieve all talks about both health and brain, then extract their lecturers*, the planner constructs a sequential program where the input texts are first filtered using two classification heads for "health" and "brain," then conditionally passed into a span extractor to identify lecturer names. This approach integrates boolean logic, ordered execution, and parameterized prompt templates into a unified representation, ensuring that downstream behavior is both interpretable and reusable across tasks.

By explicitly codifying instructions into executable task pipelines, Falconer provides two key benefits. First, the structured representation enables generalization across diverse task formulations, including multi-label classification and multi-entity extraction. Second, codification improves transparency: each decision can be traced back to a deterministic plan, bridging the gap between user intent and system actions.

Table 1 compares the planning abilities of different models. We observe that GPT-4.1 achieves high accuracy across diverse tasks, making it a strong candidate for our planner. However, performance drops on complex tasks, which we define as multi-step tasks that require intermediate execution results rather than a single fixed string (e.g., first extracting a lecturer's name, then identifying that lecturer's profession). To further probe model limits, we include a set of miscellaneous tasks specifically designed to stress-test state-of-the-art LLMs under such challenging scenarios. While models struggle in these cases, their accuracy improves substantially with in-context learning (ICL), underscoring both the difficulty of complex tasks and the effectiveness of our framework in decomposing knowledge mining objectives into well-structured subtasks.

3.5 Generator

A major challenge in adapting a lightweight metamodel to diverse knowledge mining tasks is acquiring high-quality, task-specific supervision without incurring prohibitive costs. In Falconer, we address this challenge with a generator that bridges raw corpus data and the specialized capabilities required by the metamodel. Unlike fully synthetic data produced by large language models, which can diverge from the target distribution, our generator leverages naturally occurring corpus examples to produce realistic, task-aligned supervision.

The generator operates in three stages. First, around five percent of the entire corpus is sampled to capture the authentic distribution of the domain, which is detailed in Appendix B. Second, a powerful large language model (e.g., GPT-4.1) annotates these samples according to the planner's codified task descriptions, covering subtasks such as entity extraction, classification, and relation detection. Importantly, the generator enriches naturally occurring data with high-quality labels rather than fabricating artificial inputs, ensuring statistical fidelity to the corpus. Finally, the annotated samples are used to fine-tune the metamodel, enabling it to acquire task-specific knowledge while maintaining its efficiency advantages over large models. A summary of performance gains is provided in Table 4.

In subsequent experiments, this approach demonstrates high efficiency, achieving performance comparable to or even surpassing state-of-the-art large language models while using only 5% of the original corpus. Crucially, the generator's success hinges on access to high-quality supervision, which can be readily extended to alternative sources such as carefully curated human annotations.

3.6 Metamodel: Lightweight Yet Capable Proxy

In Falconer, the metamodel serves as the central execution engine, acting as a lightweight proxy for large language models in downstream knowledge mining tasks. Instead of relying on general-purpose LLMs for every request, we adopt Cuckoo [24], which has a parameter scale comparable to RoBERTa [18], to strike a balance between efficiency and capability. This design enables Falconer to achieve near-LLM performance with substantially reduced inference cost.

Moreover, Falconer's modular architecture leverages Cuckoo not as a monolithic generalist, but as a specialized executor within a planner-driven pipeline. The planner codifies user intents into explicit, interpretable subtasks; the metamodel then executes these subtasks with high efficiency. This separation enables Falconer to exploit the SLM-first paradigm advocated by recent research [3].

Empirically, this design achieves substantial gains in both efficiency and scalability. Cuckoo requires up to $20\times$ fewer FLOPs and $1000\times$ less memory than GPT-class models, while maintaining competitive accuracy on instruction-following and span-extraction benchmarks relevant to knowledge mining. This efficiency enables Falconer to operate cost-effectively across massive corpora, supporting real-time inference even in resource-constrained environments.

4 Experiments

We evaluate Falconer on a broad spectrum of knowledge mining tasks to demonstrate that a lightweight metamodel, when coupled with our planner-generator-executor framework, can achieve

Metamodel	Dataset	64 Samples	512 Samples	GPT-4o
Cuckoo	Biology	0.42	0.45	0.27
RoBERTa-Large	Biology	0.00	0.41	0.27
Qwen3-0.6B	Biology	0.08(0.10)	0.23(0.26)	0.27
Gemma3-1B-It	Biology	0.08(0.12)	0.12(0.29)	0.27
Cuckoo	Twitter	0.19	0.43	0.35
RoBERTa-Large	Twitter	0.00	0.38	0.35
Qwen3-0.6B	Twitter	0.06(0.12)	0.28(0.33)	0.35
Gemma3-1B-It	Twitter	0.07(0.10)	0.20(0.32)	0.35
Cuckoo	Fabrication	0.20	0.32	0.38
RoBERTa-Large	Fabrication	0.00	0.17	0.38
Qwen3-0.6B	Fabrication	0.04(0.04)	0.12(0.15)	0.38
Gemma3-1B-It	Fabrication	0.00(0.00)	0.09(0.13)	0.38
Cuckoo	Wiki	0.03	0.68	0.53
RoBERTa-Large	Wiki	0.00	0.60	0.53
Qwen3-0.6B	Wiki	0.00(0.00)	0.57(0.55)	0.53
Gemma3-1B-It	Wiki	0.00(0.02)	0.43(0.45)	0.53
Cuckoo	Vehicle	0.42	0.75	0.76
RoBERTa-Large	Vehicle	0.00	0.66	0.76
Qwen3-0.6B	Vehicle	0.00(0.00)	0.60(0.63)	0.76
Gemma3-1B-It	Vehicle	0.00(0.00)	0.42(0.55)	0.76

Table 2: Results on NER Datasets with Ground Truth labels. continually pretrained models' results are given in parentheses

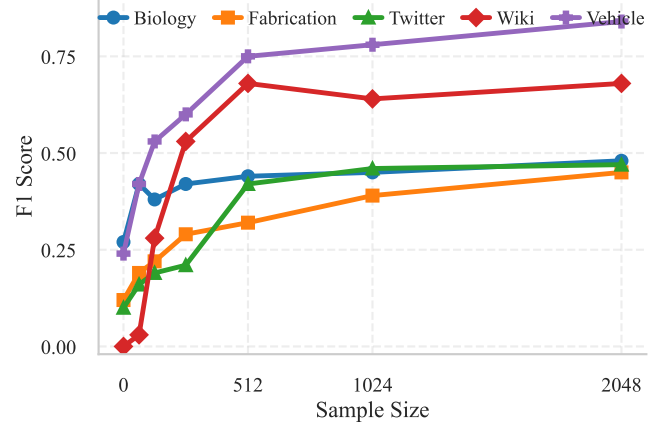


Figure 3: Model Performance under different sample size

	Model	Dataset	Basic Task				Query-based Task			Multi-entity Task			
			Task 1	Task 2	Task 3	Average	Task 1	Task 2	Average	Task 1	Task 2	Task 3	Average
0-shot	Cuckoo	TED	0.489	0.654	0.514	0.552	0.383	0.371	0.377	0.395	0.607	0.497	0.500
	Cuckoo	Steam Game	0.501	0.683	0.535	0.573	0.374	0.350	0.362	0.451	0.524	0.468	0.481
	Cuckoo	Text Message	0.584	0.694	0.585	0.621	0.418	0.392	0.405	0.564	0.583	0.530	0.559
Few-shot	Cuckoo	TED	0.658	0.758	0.683	0.699	0.532	0.557	0.545	0.644	0.692	0.661	0.666
	Roberta-Large	TED	0.552	0.587	0.553	0.564	0.446	0.511	0.479	0.517	0.566	0.531	0.538
	Qwen3-0.6B	TED	0.503	0.599	0.492	0.531	0.372	0.489	0.431	0.497	0.424	0.501	0.474
	Cuckoo	Steam Game	0.672	0.783	0.675	0.710	0.569	0.587	0.578	0.673	0.719	0.684	0.692
	Roberta-Large	Steam Game	0.509	0.525	0.517	0.517	0.434	0.452	0.443	0.588	0.383	0.564	0.512
	Qwen3-0.6B	Steam Game	0.486	0.521	0.466	0.491	0.379	0.423	0.401	0.524	0.301	0.492	0.439
	Cuckoo	Text Message	0.703	0.806	0.731	0.747	0.590	0.614	0.602	0.709	0.726	0.734	0.723
	Roberta-Large	Text Message	0.553	0.621	0.590	0.588	0.496	0.518	0.507	0.548	0.570	0.574	0.564
	Qwen3-0.6B	Text Message	0.536	0.587	0.594	0.571	0.479	0.476	0.463	0.532	0.511	0.525	0.523

Table 3: Results from various proposed tasks on 3 datasets with subtasks

performance comparable to state-of-the-art LLMs while being significantly more efficient. Our experiments are designed to answer two central questions:

- whether these metamodels maintain high alignment with human annotations on labeled datasets and
- whether Falconer can generate metamodels that faithfully approximate the behavior of large models (its annotator) on unlabeled corpora

All experiments reported in this section were conducted using a metamodel fine-tuned on 5% of the original corpus annotated by an LLM, unless otherwise specified. Model performance is evaluated using the word-level F1 score.

4.1 Labeled Dataset

This set of experiments is primarily intended to assess the consistency between the metamodel and human annotations, as well as to benchmark the performance of the metamodel against that of contemporary large language models. Furthermore, we utilized several widely adopted Named Entity Recognition (NER) datasets, including **FabNER**, **Broad Twitter**, **BC2GM**, **AnatEM**, **WikiNER**, and **FindVehicle**. These datasets were combined to construct a new benchmark, which was subsequently employed to assess the metamodel's performance across diverse groups of tasks. For particularly large datasets, such as WikiNER, we randomly sampled a subset to the mixed dataset. Meanwhile, to more explicitly illustrate the adaptability of the metamodel to downstream tasks, we present experimental results obtained by fine-tuning the metamodel with

varying amounts of training data, ranging from 64 to 2048 samples. It is worth noting that even the largest setting of 2048 samples corresponds to only 5% of the original corpus. The main results are shown in Table 2 and detailed results are plotted in Figure 3.

From the experimental results, we observe a consistent improvement in test performance as the sample size increases. Notably, the model fine-tuned with 2048 samples **outperforms GPT-4o across all task categories**, providing strong evidence of its substantial adaptability to knowledge mining tasks. Meanwhile, the rate of performance gains is closely tied to the quality of annotations generated by the large model. When the annotations are of high quality, the metamodel tends to achieve performance saturation more rapidly, as illustrated by the experiments on WikiNER. Conversely, in tasks where the large model produces suboptimal annotations, the performance of the metamodel improves more gradually, thereby reflecting the core principle of co-evolution between the metamodel and large models [24]. Finally, we also conduct experiments in controlled environments **comparing Falconer with modern agentic knowledge mining framework** which mainly utilizes small language models as executor models (e.g., Qwen3-0.6B and Gemma-1B-IT). We observe that these modern autoregressive small language models show little or no improvement over target tasks. The results demonstrate that task-specialized model designs are substantially more effective at handling knowledge tasks. More details and analysis are given in Section 5.1

4.2 Unlabeled Dataset Evaluation

To evaluate the effectiveness of Falconer in generating reliable proxy metamodels, we measure the consistency scores between the metamodel and GPT-4o across three large-scale unlabeled corpora, **TED Talk Summary, Steam Game Description, and Text Message**. We design a diverse set of knowledge mining tasks spanning three categories: **basic tasks** involving entity recognition and simple classification, **query-based tasks** requiring sentence-level semantic understanding, and **multi-label/multi-entity tasks** that demand compositional reasoning. Please refer to the complete list of tasks provided in Appendix F

Basic Task. This category benchmarks the fundamental capacity of models to discern labels, entities, and relations. We construct a suite of tasks that closely approximate real-world knowledge mining settings, exemplified by sample 1 and 2 in Appendix D. The task set spans elementary classification, entity and relation extraction, as well as composite formulations integrating both. For pairwise relation extraction tasks, we further stipulate that one entity participating in the relation is pre-specified, thereby isolating the model’s ability to infer the remaining relational structure. As shown in Table 3, the tasks categorized as Basic Task demonstrate that, after fine-tuning, the metamodel consistently achieves high agreement with the large model.

Query-Based Task. This category of tasks focuses on assessing the model’s ability to capture more complex sentence-level semantics, as exemplified by sample 3 and 4 in Appendix D. Illustrated in Table 3, the corresponding tasks are represented by Query-based Task. With appropriate fine-tuning, the metamodel demonstrates competitive performance on complex tasks. It is worth noting that

the untuned metamodel exhibits the weakest performance in this category; however, fine-tuning yields substantial improvements. For instance, given the task prompt “retrieve all texts that are primarily about medicine, and extract what the lecturer will talk about”, the initial metamodel achieves an F1 score of only 0.23 when compared against GPT-4o as the reference. After fine-tuning with only a small fraction of the annotated corpus, its F1 score increases to 0.56. These results highlight the model’s strong capacity to adapt effectively to downstream tasks.

Multi-entity Task. This category of tasks extends metamodel evaluation to multi-label and multi-entity scenarios (sample 5 in Appendix D). Prior work highlights the limitations of large language models in multi-label classification [20, 40]. In contrast, our framework employs the planner to decompose such tasks into sequential subtasks, whose outputs are aggregated to form the final result. For instance, the query “retrieve all speeches concerning both health and the brain” is decomposed into two classification subtasks—health-related and brain-related—whose results are combined via Boolean logic. This structured decomposition enables logically consistent and accurate performance in multi-label classification and multi-entity extraction.

The experimental results for Multi-entity Task, as reported in Table 3, indicate that the adapted metamodel demonstrates strong proficiency in handling multi-entity tasks, achieving performance that is competitive with, and in some cases surpasses, results obtained through multi-turn prompting augmented with human annotations.

5 Analysis

5.1 Ablation on MetaModel

In our experiments, we observe that pipelines employing modern state-of-the-art small language models as executor models exhibit significantly worse generalization performance on downstream tasks compared to pipelines based on small models trained from RoBERTa architecture and the Next Token Extraction(NTE) paradigm[24]. More generally, we hypothesize that autoregressive models are not an optimal choice for general-purpose knowledge mining tasks, although it is always the case in other modern agentic framework[13, 14]. To further investigate the behavior of autoregressive models in knowledge mining, we continually training Qwen3-0.6B and Gemma-1B-IT on the same NER pretraining dataset as Cuckoo, obtaining a new set of models. These datasets are labeled with target spans typical of NER tasks and encompass diverse domains, including biology, dialogue, healthcare etc which provide informative signals for knowledge mining tasks. As shown in Table 2, the continually pretrained models obtain little or no improvement over evaluation tasks. This suggests that autoregressive models are not effective at extracting knowledge from NER-style supervision in the training data, highlighting the need for task-specialized designs.

5.2 Continual Integration Analysis

In Table 4, fine-tuning was restarted from a fresh base model for each task. In practice, however, continual learning is equally important, as models are expected to sustain performance across sequential tasks while retaining competence from earlier ones. To evaluate

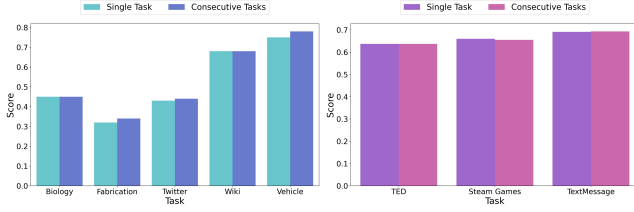


Figure 4: Performance on labeled datasets for single-task training with a new metamodel and consecutive-task training with the metamodel from the previous task. Performance on unlabeled datasets under the same settings.

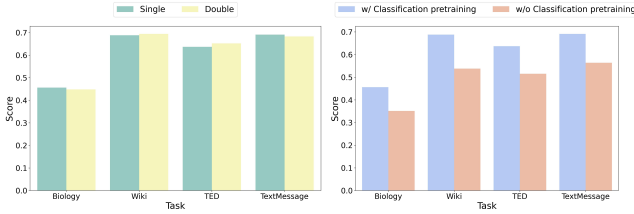


Figure 5: Performance with varying numbers of metamodels across different task types, and comparison of different pre-training strategies.

Models	F1 score
Roberta-Large (degraded data)	0.24
Roberta-Large (original data)	0.40
Cuckoo (degraded data)	0.41
Cuckoo (original data)	0.42
Annotator (GPT-4o)	0.27

Figure 6: Results of different models on Biology Task

this ability, we reformulated the setup into a sequence of five tasks, where each task used the model fine-tuned on its predecessor as the base. We report the results in Figure 4, averaging over subtasks when applicable. The figure demonstrates the metamodel’s performance under sequential fine-tuning and evaluation on consecutive tasks.

We observe that models undergoing multiple rounds of fine-tuning on sequential tasks maintain capabilities comparable to those fine-tuned directly from the base model. Overall, our evaluation highlights the metamodel’s continual integration ability, demonstrating its effectiveness in sustaining high performance across a broad spectrum of real-world tasks. Moreover, the results validate that the proposed framework substantially alleviates the deployment overhead associated with adapting models to diverse tasks.

In this section, we further highlight the efficiency and performance advantages of our framework. While prior experiments benchmarked RoBERTa-large against multiple baselines, its lack of inherent instruction-following ability required training two task-specific variants for classification and extraction. By contrast, our

framework enables the incubation of a single metamodel that leverages instruction-following to generalize across heterogeneous tasks. To validate this, we additionally trained two separate metamodels—one for classification and one for extraction—on the same benchmarks. As shown in left panel of Figure 5, their performance is nearly indistinguishable from that of a unified model, underscoring that a single metamodel can achieve state-of-the-art performance across task types while significantly reducing deployment overhead.

Meanwhile, to further substantiate the metamodel’s capacity for continual generalization across novel tasks, we additionally evaluate its performance without pretraining on the classification dataset (detailed in Section 3.3). This comparison highlights the model’s adaptability, demonstrating its ability to rapidly generalize to unseen tasks through a combination of pretraining and fine-tuning. As shown in right panel of Figure 5, we fine-tune the model on datasets of equal size and train for the same number of epochs to ensure a controlled setting. The results indicate that the pretrained metamodel achieves significantly faster convergence when adapted to new tasks, underscoring its strong generalization and adaptability in continual learning scenarios.

5.3 Case Study: Arising Abilities of model

Task: Extract all gene names in the give text

In the course of Hepatitis A HBs - and HBe - antigen as well as HBc (IgM and IgG) - , HBs - and HBe - antibodies can be detected .

Answers:

GPT-4o:['None'] **Untuned model:**['None']

Tuned model:['HBs', 'HBe', 'HBc']

Table 4 reveals the model’s strong performance across tasks, with notable patterns emerging. On **Biology** tasks, GPT-4o achieves an average F1 of 0.27—barely matching the metamodel’s zero-shot performance—highlighting the low quality of GPT-4o annotations. Intriguingly, fine-tuning the metamodel on these noisy labels still yields substantial gains. Manual analysis attributes 74% of this improvement to the phenomenon illustrated in 5.3, which we term **arising abilities**.

As shown in 5.3, we define arising abilities as **the model’s capacity to correct its outputs even when provided with inaccurate annotation guidance from contemporary LLMs**. Similar phenomena have been observed in prior studies [28, 45], which report that models can self-correct under random or deliberately misleading guidance. These works attribute this capability to the elicitation of the model’s extensive pretrained knowledge, aligning with our analytical interpretation. To further validate this hypothesis, we conducted a series of controlled experiments, detailed below.

The metamodel’s pretraining on IE tasks [24], which encode entities with positional information, appears to endow it with a strong sensitivity to token structure. We hypothesize that this enables the model to spontaneously extract entities at corresponding positions when faced with new entities sharing similar positional patterns. To test this, we degraded GPT-annotated data by randomizing span

start positions while preserving span endings, retaining primarily positional cues. For instance, a consecutive span: *cytochrome P-450 monooxygenase* could be annotated as **cytochrome P-450 monooxygenase**, **cytochrome P-450 monooxygenase** or **cytochrome P-450 monooxygenase**. Fine-tuning on this degraded data yielded performance nearly identical to training on the original annotations, whereas RoBERTa-large suffered a substantial drop (Table 6). These results suggest that the model’s arising ability is driven almost entirely by positional supervision, revealing a striking capability arising from its pretraining knowledge.

6 Conclusion

This paper proposes a framework for the automated execution of knowledge mining tasks, which decomposes each task into several subtasks and employs a unified model to perform them. Consequently, users only need to provide a task prompt and specify the output format to effortlessly execute a wide range of knowledge mining tasks, while benefiting from performance surpassing that of even the most power modern large language models, as well as 90% inference costs decrease and 20x inference speed increase.

References

- [1] Monica Agrawal, Stefan Hegselmann, Hunter Lang, Yoon Kim, and David A. Sontag. 2022. Large language models are few-shot clinical information extractors. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.). Association for Computational Linguistics, 1998–2022. doi:10.18653/V1/2022.EMNLP-MAIN.130
- [2] Anthropic. 2025. Claude 3.7 Sonnet and Claude Code. <https://www.anthropic.com/news/claude-3-7-sonnet>
- [3] Peter Belcak, Greg Heinrich, Shizhe Diao, Yonggan Fu, Xin Dong, Saurav Muradharan, Yingyan Celine Lin, and Pavlo Molchanov. 2025. Small Language Models are the Future of Agentic AI. *arXiv preprint arXiv:2506.02153* (2025).
- [4] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. 2024. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38, 17682–17690.
- [5] Sergei Bogdanov, Alexandre Constantin, Timothée Bernard, Benoît Crabbé, and Etienne Bernard. 2024. NuNER: Entity Recognition Encoder Pre-training via LLM-Annotated Data. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, 11829–11841. <https://aclanthology.org/2024.emnlp-main.660>
- [6] Jack Boylan, Chris Hokamp, and Demian Gholipour Ghalandari. 2025. GLiREL-Generalist Model for Zero-Shot Relation Extraction. *arXiv preprint arXiv:2501.03172* (2025).
- [7] Xavier Carreras and Lluís Màrquez. 2005. Introduction to the CoNLL-2005 shared task: Semantic role labeling. In *Proceedings of the ninth conference on computational natural language learning (CoNLL-2005)*, 152–164.
- [8] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128* (2023).
- [9] Gheorghe Comanici, Eric Bieber, Mike Schaeckermann, Ice Pasupat, Naveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. *arXiv preprint arXiv:2507.06261* (2025).
- [10] Ning Ding, Guangwei Xu, Yulin Chen, Xiaobin Wang, Xu Han, Pengjun Xie, Haitao Zheng, and Zhiyuan Liu. 2021. Few-NERD: A Few-shot Named Entity Recognition Dataset. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2021, (Volume 1: Long Papers), Virtual Event, August 1-6, 2021*, Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (Eds.). Association for Computational Linguistics, 3198–3213. doi:10.18653/V1/2021.ACL-LONG.248
- [11] Zane Durante, Qiuyuan Huang, Naoki Wake, Ran Gong, Jae Sung Park, Bidipta Sarkar, Rohan Taori, Yusuke Noda, Demetri Terzopoulos, Yejin Choi, et al. 2024. Agent ai: Surveying the horizons of multimodal interaction. *arXiv preprint arXiv:2401.03568* (2024).
- [12] Yao Fu, Dong-Ki Kim, Jaekyeom Kim, Sungryull Sohn, Lajanugen Logeswaran, Kyunghoon Bae, and Honglak Lee. 2024. Autoguide: Automated generation and selection of context-aware guidelines for large language model agents. *arXiv preprint arXiv:2403.08978* (2024).
- [13] Jian Guan, Wei Wu, Peng Xu, Hongning Wang, Minlie Huang, et al. 2024. Amor: A recipe for building adaptable modular knowledge agents through process feedback. *Advances in Neural Information Processing Systems* 37 (2024), 126118–126148.
- [14] Sirui Hong, Xiaowu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. 2023. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352* 3, 4 (2023), 6.
- [15] Moritz Lauer, Wouter van Atteveldt, Andreu Casas, and Kasper Welbers. 2023. Building efficient universal classifiers with natural language inference. *arXiv preprint arXiv:2312.17543* (2023).
- [16] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [17] Yongqi Li, Yu Yu, and Tiejun Qian. 2023. Type-Aware Decomposed Framework for Few-Shot Named Entity Recognition. In *Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, December 6-10, 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, 8911–8927. doi:10.18653/V1/2023.FINDINGS-EMNLP.598
- [18] Yinhao Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
- [19] Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. 2023. Chameleon: Plug-and-play compositional reasoning with large language models. *Advances in Neural Information Processing Systems* 36 (2023), 43447–43478.
- [20] Marcus Ma, Georgios Chochlakis, Niyantha Maruthu Pandiyan, Jesse Thomason, and Shrikanth Narayanan. 2025. Large Language Models Do Multi-Label Classification Differently. *arXiv preprint arXiv:2505.17510* (2025).
- [21] Mingyu Derek Ma, Xiaoxuan Wang, Po-Nien Kung, P Jeffrey Brantingham, Nanyun Peng, and Wei Wang. 2024. STAR: boosting low-resource information extraction by structure-to-text data generation with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38, 18751–18759.
- [22] OpenAI. 2025. GPT-5 System Card. *Blog* (2025).
- [23] Baolin Peng, Michel Galley, Pengcheng He, Hao Cheng, Yujia Xie, Yu Hu, Qiuyuan Huang, Lars Liden, Zhou Yu, Weizhu Chen, et al. 2023. Check your facts and try again: Improving large language models with external knowledge and automated feedback. *arXiv preprint arXiv:2302.12813* (2023).
- [24] Letian Peng, Zilong Wang, Feng Yao, and Jingbo Shang. 2025. Cuckoo: An IE Free Rider Hatched by Massive Nutrition in LLM’s Nest. *arXiv preprint arXiv:2502.11275* (2025).
- [25] Letian Peng, Zilong Wang, Feng Yao, Zihan Wang, and Jingbo Shang. 2024. MetaIE: Distilling a Meta Model from LLM for All Kinds of Information Extraction Tasks. *CoRR abs/2404.00457* (2024). arXiv:2404.00457 doi:10.48550/ARXIV.2404.00457
- [26] Maria Pontiki, Dimitrios Galanis, Haris Papageorgiou, Ion Androutsopoulos, Suresh Manandhar, Mohammad Al-Smadi, Mahmoud Al-Ayyoub, Yanyan Zhao, Bing Qin, Orphée De Clercq, et al. 2016. Semeval-2016 task 5: Aspect based sentiment analysis. In *International workshop on semantic evaluation*, 19–30.
- [27] Erik F Sang and Fien De Meulder. 2003. Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition. *arXiv preprint cs/0306050* (2003).
- [28] Rulin Shao, Shuyue Stella Li, Rui Xin, Scott Geng, Yiping Wang, Sewoong Oh, Simon Shaoqi Du, Nathan Lambert, Sewon Min, Ranjay Krishna, et al. 2025. Spurious rewards: Rethinking training signals in rlvr. *arXiv preprint arXiv:2506.10947* (2025).
- [29] Tao Shen, Guodong Long, Xiubo Geng, Chongyang Tao, Tianyi Zhou, and Daxin Jiang. 2023. Large language models are strong zero-shot retriever. *arXiv preprint arXiv:2304.14233* (2023).
- [30] Wenqi Shi, Ran Xu, Yuchen Zhuang, Yue Yu, Jieyu Zhang, Hang Wu, Yuanda Zhu, Joyce Ho, Carl Yang, and May D Wang. 2024. Ehragent: Code empowers large language models for few-shot complex tabular reasoning on electronic health records. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing. Conference on Empirical Methods in Natural Language Processing*, Vol. 2024, 22315.
- [31] Noah Shinn, Federico Cassano, Beck Labash, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning, 2023. URL <https://arxiv.org/abs/2303.11366> (2023).
- [32] Simone Tedeschi and Roberto Navigli. 2022. MultiNERD: A Multilingual, Multi-Genre and Fine-Grained Dataset for Named Entity Recognition (and Disambiguation). In *Findings of the Association for Computational Linguistics: NAACL 2022*,

- Seattle, WA, United States, July 10-15, 2022, Marine Carpuat, Marie-Catherine de Marneffe, and Iván Vladimir Meza Ruiz (Eds.). Association for Computational Linguistics, 801–812. doi:10.18653/V1/2022.FINDINGS-NAACL.60
- [33] Christopher Walker, Stephanie Strassel, Julie Medero, and Kazuaki Maeda. 2006. ACE 2005 Multilingual Training Corpus. Web Download. doi:10.35111/mwxc-vh88 LDC Catalog No. LDC2006T06.
- [34] Christopher Walker, Stephanie Strassel, Julie Medero, and Kazuaki Maeda. 2006. Ace 2005 multilingual training corpus. (*No Title*) (2006).
- [35] Liang Wang, Nan Yang, and Furu Wei. 2023. Query2doc: Query expansion with large language models. *arXiv preprint arXiv:2303.07678* (2023).
- [36] Shuhe Wang, Xiaofei Sun, Xiaoya Li, Rongbin Ouyang, Fei Wu, Tianwei Zhang, Jiwei Li, and Guoyin Wang. 2023. GPT-NER: Named Entity Recognition via Large Language Models. *CoRR abs/2304.10428* (2023). arXiv:2304.10428 doi:10.48550/ARXIV.2304.10428
- [37] Zilong Wang, Zifeng Wang, Long T. Le, Huaixiu Steven Zheng, Swaroop Mishra, Vincent Perot, Yuwei Zhang, Anush Mattapalli, Ankur Taly, Jingbo Shang, Chen-Yu Lee, and Tomas Pfister. 2025. Speculative RAG: Enhancing Retrieval Augmented Generation through Drafting. In *Proceedings of the International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=xgQfWbV6Ey> Poster.
- [38] Shirley Wu, Shiyu Zhao, Qian Huang, Kexin Huang, Michihiro Yasunaga, Kaidi Cao, Vassilis Ioannidis, Karthik Subbian, Jure Leskovec, and James Y Zou. 2024. Avatar: Optimizing llm agents for tool usage via contrastive reasoning. *Advances in Neural Information Processing Systems* 37 (2024), 25981–26010.
- [39] Derong Xu, Wei Chen, Wenjun Peng, Chao Zhang, Tong Xu, Xiangyu Zhao, Xian Wu, Yefeng Zheng, Yang Wang, and Enhong Chen. 2024. Large language models for generative information extraction: a survey. *Frontiers Comput. Sci.* 18, 6 (2024), 186357. doi:10.1007/S11704-024-40555-Y
- [40] Hanzhi Xu, Renze Lou, Jiangshu Du, Vahid Mahzoon, Elmira Talebianaraki, Zhuoan Zhou, Elizabeth Garrison, Slobodan Vucetic, and Wenpeng Yin. 2024. LLMs' Classification Performance is Overclaimed. *arXiv preprint arXiv:2406.16203* (2024).
- [41] Lu Xu, Hao Li, Wei Lu, and Lidong Bing. 2020. Position-aware tagging for aspect sentiment triplet extraction. *arXiv preprint arXiv:2010.02609* (2020).
- [42] Xiaolong Xu, Yibo Zhou, Haolong Xiang, Xiaoyong Li, Xuyun Zhang, Lianying Qi, and Wanchun Dou. [n. d.]. DocKS-RAG: Optimizing Document-Level Relation Extraction through LLM-Enhanced Hybrid Prompt Tuning. In *Forty-second International Conference on Machine Learning*.
- [43] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.
- [44] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models, 2023. URL <https://arxiv.org/abs/2210.03629> (2023).
- [45] Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. 2025. Limo: Less is more for reasoning. *arXiv preprint arXiv:2502.03387* (2025).
- [46] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. Expel: Llm agents are exponential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 19632–19642.
- [47] Shengyao Zhuang, Bing Liu, Bevan Koopman, and Guido Zuccon. 2023. Open-source large language models are strong zero-shot query likelihood models for document ranking. *arXiv preprint arXiv:2310.13243* (2023).
- [48] Chang Zong, Yuchen Yan, Weiming Lu, Jian Shao, Eliot Huang, Heng Chang, and Yueting Zhuang. 2024. Triad: A Framework Leveraging a Multi-Role LLM-based Agent to Solve Knowledge Base Question Answering. *arXiv preprint arXiv:2402.14320* (2024).

A Cuckoo For Text classification

Prompt

User:

Choices:

yes

no

Input Text Question: Based on above sentence, is the following sentence true or not ?

This text is about **label**

Assistant:

Answer:

We adopt the aforementioned template and leverage the token-level supervision provided by Cuckoo to reformulate the classification task into a more general natural language inference (NLI) problem. An illustrative example is provided in Figure 2.

B Generating Fine-tuning Samples

We leverage the metamodel’s inherent pretraining knowledge and adopt a heuristic approach to obtain a relatively high-quality fine-tuning dataset. For classification tasks, the generation of fine-tuning samples is illustrated in Algorithm 1, whereas for extraction tasks, we directly employ random sampling.

Input: Corpus C , label l , sample size N

Output: Training set $\mathcal{T}$

Initialize empty set $\mathcal{T}$;

foreach sample $x \in C$ **do**

 Compute score $s(x, l)$ using metamodel;

end

Sort all samples in C by score $s(x, l)$ in descending order;

Select top N samples $\{x_1^+, \dots, x_N^+\}$ as positive set $\mathcal{P}$;

Select bottom N samples $\{x_1^-, \dots, x_N^-\}$ as negative set $\mathcal{N}$;

Construct training set $\mathcal{T} = \mathcal{P} \cup \mathcal{N}$;

return $\mathcal{T}$;

Algorithm 1: Classification Training Set Generation

C Continual Pretraining on autoregressive models

Metamodel	Dataset	64 Samples	512 Samples	GPT-4o
Qwen3-0.6B	Biology	0.08	0.258	0.27
Gemma3-1B-It	Biology	0.08	0.12	0.27
Qwen3-0.6B	Twitter	0.06	0.28	0.35
Gemma3-1B-It	Twitter	0.07	0.20	0.35
Qwen3-0.6B	Fabrication	0.04	0.55	0.38
Gemma3-1B-It	Fabrication	0.00	0.09	0.38
Qwen3-0.6B	Wiki	0.00	0.55	0.53
Gemma3-1B-It	Wiki	0.00	0.43	0.53
Qwen3-0.6B	Vehicle	0.00	0.60	0.76
Gemma3-1B-It	Vehicle	0.00	0.42	0.76

Table 4: Results on NER Datasets with Ground Truth labels

D Sample Task

Sample Task

1. retrieve all speaks which is mainly about finance and extract its lecturer
2. extract all locations mentioned in the text
3. find all talks that address breaking gender stereotypes in modern society, and include all countries mentioned

4. retrieve all speaks which is mainly about how mental health influences our daily lives and extract all the institution name mentioned
5. retrieve all speaks which is mainly about both health and brain in the speak, then extract their lecturer

E Sample Planning Code

```
def GPT_pipeline(Input_Corpus):
    labels = ['finance']
    label_results = get_label(Input_Corpus, labels)

    finance_indices = [i for i, result in enumerate(label_results) if result[0].lower() == 'yes']
    filtered_texts = [Input_Corpus[i] for i in finance_indices]
    if not filtered_texts:
        return []
    instruction_spans = "Extract the lecturer of the speak in the given text."
    spans_results = get_spans(filtered_texts, instruction_spans)
    output = []
    for idx, orig_idx in enumerate(finance_indices):
        output.append({
            'text': Input_Corpus[orig_idx],
            'spans': spans_results[idx]
        })
    return output
```

F Human Proposed Task on Unlabeled Datasets

Tasks on TED description Dataset

1. retrieve all speaks which is mainly about finance and extract its lecturer
2. output all speaks which is mainly about mental health and extract its speakers
3. return all speaks which is mainly about environment and extract all the locations mentioned in the text
4. retrieve talks whose main theme is artificial intelligence and list all professions mentioned
5. get all talks that center on medicine and identify all disease mentioned
6. collect all speaks which is mainly about finance
7. give out all speaks which is mainly about health
8. retrieve all speaks which is mainly about education
9. gather all speaks which is mainly about technology
10. output all speaks which is mainly about politics
11. Extract all locations mentioned
12. Extract all time mentioned
13. Extract all countries mentioned
14. Extract all website mentioned
15. Extract all person mentioned
16. retrieve all speaks which is mainly about how artificial intelligence could affect our lives and its lecturer
17. gather talks that mainly discuss climate change and its global impact, and provide all countries mentioned
18. retrieve all speaks which is mainly about how mental health influences our daily lives and extract all the institution name mentioned
19. find talks that analyze the future of work in an automated world, and return the occupation of the lecturer

20. get all talks that address breaking gender stereotypes in modern society, and include the lecturer
21. retrieve all texts which is mainly about medicine, and extract what the lecturer will talk about
22. retrieve all texts which are mainly about health, and extract all the disease and its associated cause
23. find all texts which are mainly about literature, and extract all the awards of [PERSON]
24. find all texts which are mainly about science, and extract the profession of [PERSON]
25. output all texts which are mainly about history, and extract all the events and the time of the events
26. retrieve all speaks which is mainly about both health and brain in the speak, then extract their lecturer
27. retrieve all speaks which is mainly about both design and creativity in the speak, then extract all artists mentioned
28. retrieve all speaks which is mainly about both medicine and surgery in the speak, then extract all countries mentioned
29. retrieve all speaks which is mainly about artificial intelligence and ethics in the speak, then extract all location mentioned
30. retrieve all speaks which is mainly about artificial intelligence and machine learning in the speak, then extract the lecturer
31. gather all texts which is mainly about finance or artificial intelligence, and extract the lecturer
32. get all texts which is mainly about education or biology, and extract all professions
33. return all texts which is mainly about philosophy or literature, and extract all person mentioned
34. output all speaks which centers on literature or philosophy, then extract all the university affiliation
35. retrieve all speaks which centers on music or visual arts, then extract the awards
36. retrieve all speaks which is mainly about health but is not about brain, then extract their lecturer
37. retrieve all texts which is mainly about environment but is not about climate change, and extract the locations
38. identify all talks mainly focusing on finance but not mentioning technology, then extract all lecturer name mentioned
39. find all speeches mainly about artificial intelligence but without any reference to machine learning, then list all researchers mentioned
40. filter all talks centered on technological innovation but not mentioning blockchain, and extract all numbers mentioned